ใบงานการใช้ **Node.js** ร่วมกับ **PostgreSQL** สำหรับการแสดงผลข้อมูลในแผนภูมิ

รูปแบบกราฟเส้นและกราฟแท่ง คุณสามารถใช้ไลบรารีอย่าง EJS (Embedded JavaScript Templates) หรือ Handlebars เพื่อแสดงผลข้อมูลในฝั่งเซิร์ฟเวอร์ และใช้ Chart.js เพื่อแสดงกราฟ รวมถึงการจัดการ DataGrid ด้วย HTML และ JavaScript ธรรมดา

1. สร้างฐานข้อมูล PostgreSQL

สร้างฐานข้อมูลและตารางใน PostgreSQL ที่จะใช้เก็บข้อมูล:

```
CREATE DATABASE graphdb;

\c graphdb


CREATE TABLE sales_data (
    id SERIAL PRIMARY KEY,
    date DATE NOT NULL,
    sales INTEGER NOT NULL
);


INSERT INTO sales_data (date, sales) VALUES
('2024-01-01', 150),
('2024-01-02', 200),
('2024-01-03', 180),
('2024-01-04', 220),
('2024-01-05', 170);
```

2. สร้างโปรเจค Node.js

เปิด terminal และสร้างโปรเจค Node.js ใหม่:

mkdir node-express-postgres-chart

cd node-express-postgres-chart

npm init -y

npm install express pg ejs

3. ตั้งค่าโครงสร้างโปรเจค

สร้างโครงสร้างไฟล์ดังนี้

node-express-postgres-chart/

```
│
├── server.js           # ไฟล์หลักของ Node.js
├── views/
│   └── index.ejs       # หน้าแสดงผล (ใช้ EJS สำหรับ templating)
└── public/
    └── js/
        └── chart.js    # ไฟล์ JavaScript สำหรับ Chart.js
```

4. เขียนโค้ดใน server.js

โค้ดสำหรับสร้างเซิร์ฟเวอร์ Express และเชื่อมต่อกับ PostgreSQL

```
const express = require('express');

const { Pool } = require('pg');

const path = require('path');


const app = express();


// ตั้งค่า EJS เป็น view engine

app.set('view engine', 'ejs');

app.set('views', path.join(__dirname, 'views'));


// เสิร์ฟไฟล์ static

app.use(express.static(path.join(__dirname, 'public')));


// ตั้งค่าการเชื่อมต่อ PostgreSQL

const pool = new Pool({

    user: 'yourusername',

    host: 'localhost',

    database: 'graphdb',

    password: 'yourpassword',
```

```
    port: 5432,
});

// เส้นทางหลัก
app.get('/', async (req, res) => {
    try {
        const result = await pool.query('SELECT * FROM sales_data ORDER BY date');
        const salesData = result.rows;

        // ส่งข้อมูลไปยังหน้า index.ejs
        res.render('index', { salesData });
    } catch (err) {
        console.error(err);
        res.status(500).send('Server error');
    }
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
    console.log(`Server running on port ${PORT}`);
});
```

5. เขียนโค้ดใน views/index.ejs

ไฟล์ EJS สำหรับแสดง DataGrid และกราฟ

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Sales Data</title>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
</head>
<body>
    <h1>Sales Data</h1>

    <!-- DataGrid -->
    <table border="1" id="data-grid">
        <thead>
            <tr>
                <th>ID</th>
                <th>Date</th>
                <th>Sales</th>
            </tr>
```

```
    </thead>
    <tbody>
        <% salesData.forEach(function(sale) { %>
            <tr data-id="<%= sale.id %>" class="data-row">
                <td><%= sale.id %></td>
                <td><%= sale.date %></td>
                <td><%= sale.sales %></td>
            </tr>
        <% }); %>
    </tbody>
</table>

<!-- Dropdown สำหรับเปลี่ยนกราฟ -->
<select id="chart-type">
    <option value="line">Line Chart</option>
    <option value="bar">Bar Chart</option>
</select>

<!-- Canvas สำหรับแสดงกราฟ -->
<canvas id="salesChart"></canvas>
```

```
    <script src="/js/chart.js"></script>
</body>
</html>
```

6. เขียนโค้ดใน public/js/chart.js

ไฟล์ JavaScript สำหรับจัดการการแสดงกราฟและ DataGrid

```
// ดึงข้อมูลจาก table มาแสดงผลในกราฟ
document.addEventListener('DOMContentLoaded', () => {
    const ctx = document.getElementById('salesChart').getContext('2d');
    const chartTypeSelector = document.getElementById('chart-type');
    let chart;

    // ฟังก์ชันสำหรับสร้างข้อมูลกราฟ
    const getChartData = () => {
        const labels = [];
        const sales = [];
        document.querySelectorAll('#data-grid tbody tr').forEach(row => {
            labels.push(row.cells[1].textContent);  // ดึงวันที่จากแถว
            sales.push(parseInt(row.cells[2].textContent));  // ดึงยอดขายจากแถว
        });
        return { labels, sales };
```

```
};

// ฟังก์ชันสำหรับอัพเดทกราฟ
const updateChart = (chartType) => {
    if (chart) chart.destroy();  // ลบกราฟเดิมก่อนสร้างใหม่

    const chartData = getChartData();
    chart = new Chart(ctx, {
        type: chartType,
        data: {
            labels: chartData.labels,
            datasets: [{
                label: 'Sales',
                data: chartData.sales,
                borderColor: 'rgba(75, 192, 192, 1)',
                backgroundColor: 'rgba(75, 192, 192, 0.2)',
                borderWidth: 1,
            }]
        },
        options: {
            scales: {
```

```
                    y: {
                        beginAtZero: true
                    }
                }
            }
        });
    };

    // สร้างกราฟเริ่มต้นเป็นกราฟเส้น
    updateChart('line');

    // เมื่อมีการเปลี่ยนชนิดกราฟ
    chartTypeSelector.addEventListener('change', (e) => {
        updateChart(e.target.value);
    });
});
```

7. รันเซิร์ฟเวอร์

หลังจากตั้งค่าโปรเจคเรียบร้อยแล้ว คุณสามารถรันเซิร์ฟเวอร์ได้โดยใช้คำสั่ง

```
node server.js
```

จากนั้นเปิดเบราว์เซอร์และไปที่ http://localhost:3000 จะเห็นตารางข้อมูล (DataGrid) และกราฟแสดงยอดขายในรูปแบบกราฟเส้น (Line Chart) โดยคุณสามารถเลือกเปลี่ยนเป็นกราฟแท่ง (Bar Chart) ได้จาก Dropdown ด้านบน

สรุป

1. เราได้สร้าง Node.js เซิร์ฟเวอร์โดยใช้ Express.js สำหรับเชื่อมต่อกับ PostgreSQL
2. ใช้ EJS สำหรับการแสดงผลหน้า HTML โดยส่งข้อมูลจากฐานข้อมูลไปแสดง
3. ใช้ Chart.js สำหรับการสร้างกราฟ และจัดการการเปลี่ยนแปลงประเภทกราฟ
4. ใช้ JavaScript ในฝั่ง client เพื่อทำให้กราฟเปลี่ยนได้ตามการเลือกจาก DataGrid